Universidad Autónoma de Tamaulipas
Facultad de Ingeniería "Arturo Narro Siller"
Programación Avanzada — Grupo K

Reporte de Laboratorio - Práctica 3: Consumer Loan Assistant (JavaFX, FXML y MVC)

Alumno: Lopez Cruz Emiliano
Matrícula: a2253332354 | **Grupo:** K
Materia: Programación Avanzada
Proyecto en Eclipse: PA_a2253332354_Practica3
Tema: Adaptación del proyecto Consumer Loan Assistant de Swing a JavaFX con FXML y patrón MVC

1. Explicación de la arquitectura MVC utilizada

El programa original *Consumer Loan Assistant* del capítulo 3 venía programado en una sola clase `LoanAssistant extends JFrame` donde el constructor posicionaba cada control con `GridBagConstraints` y en el mismo archivo se hacían los cálculos matemáticos de anualidades, logaritmos, validaciones de números decimales y cuadros de diálogo `JOptionPane`.

En el proyecto `PA_a2253332354_Practica3` se reorganizó toda la aplicación en cuatro paquetes siguiendo el patrón MVC:

- **Vista (`vista/VistaAsistentePrestamo.fxml`):** Sustituye el `GridBagLayout` hecho por código en Swing por un `GridPane` en FXML de 4 columnas y 6 filas, respetando las fuentes (Arial 16 para etiquetas y campos de entrada, Arial Bold 14 para botones y Courier New 14 para el área de análisis) y alineando los números a la derecha (`CENTER_RIGHT`).
- **Modelo (`modelo/Prestamo.java` y `modelo/CalculadoraPrestamo.java`):** `Prestamo` almacena los datos del crédito (saldo, tasa anual, número de meses, pago mensual, pago final, total pagado e intereses) como propiedades de JavaFX. Por su parte, `CalculadoraPrestamo` concentra todas las fórmulas financieras del documento original, incluyendo el caso especial cuando la tasa de interés es `0%` y el ciclo que calcula el ajuste del último pago (`finalPayment`).
- **Controlador (`controlador/ControladorAsistentePrestamo.java`):** Conecta los componentes del FXML mediante `@FXML`. Se encarga de cambiar entre los dos modos de operación con los botones **X** (`monthsButton` y `paymentButton`), pintar de amarillo claro (`#FFFF80`) el campo que se calcula automáticamente quitándole el foco de tabulación (`setFocusTraversable(false)`), pasar el foco al siguiente campo cuando se presiona la tecla `<Enter>` y alternar la activación entre **Compute** y **New Loan Analysis**.
- **Utilerías (`utilerias/Validador.java` y `utilerias/Dialogos.java`):** En `Validador` se puso el método `validateDecimalNumber(TextField tf)` que revisa carácter por carácter que sólo haya dígitos y a lo más un punto decimal, devolviendo el foco al campo si hay un error. En `Dialogos` se programaron las ventanas de aviso y la pregunta de confirmación de pago mínimo usando `Alert` de JavaFX.

2. Descripción de las clases desarrolladas

- **`application.Main`:** Carga `VistaAsistentePrestamo.fxml`, fija el título de la ventana como *"Loan Assistant"*, bloquea el cambio de tamaño (`setResizable(false)`) y la centra en la pantalla.
- **`modelo.Prestamo`:** Entidad con propiedades `DoubleProperty` e `IntegerProperty` para guardar los datos de entrada y los resultados del análisis.
- **`modelo.CalculadoraPrestamo`:**
  - `calcularPagoMensual(balance, interesAnualPct, months)`: Calcula el interés mensual dividiendo entre 1200. Si el interés es 0, divide `balance / months`; de lo contrario aplica la fórmula con `multiplier = Math.pow(1 + monthlyInterest, months)`.
  - `calcularNumeroPagos(balance, interesAnualPct, payment)`: Si el interés es 0, calcula `(int)(balance / payment)`; si es mayor a 0 usa logaritmos naturales con `Math.log`.
  - `obtenerPagoMinimo(balance, interesAnualPct)`: Calcula `balance * monthlyInterest + 1.0` para evitar la amortización negativa cuando se calcula el número de pagos.
  - `generarAnalisisPrestamo(Prestamo)`: Simula los primeros `months - 1` pagos sumando intereses y restando la mensualidad; si el saldo restante rebasa el pago mensual, aplica un pago más e incrementa `months`, devolviendo el texto formateado a dos decimales para el recuadro de análisis.
- **`utilerias.Validador`:** Implementa `validateDecimalNumber` adaptado al control `javafx.scene.control.TextField`.
- **`utilerias.Dialogos`:** Contiene `mostrarErrorEntrada` (equivalente al `JOptionPane` con botón OK), `confirmarPagoMinimo` (con botones `YES` y `NO`) y `guardarAnalisisTxt` para dejar registro de las corridas en `HistorialAnalisisPrestamos.txt`.
- **`controlador.ControladorAsistentePrestamo`:** Administra todos los eventos `onAction` definidos en el FXML (`computeButtonActionPerformed`, `newLoanButtonActionPerformed`, `monthsButtonActionPerformed`, `paymentButtonActionPerformed`, `exitButtonActionPerformed` y el paso de foco con Enter en los cuatro `TextField`).

3. Capturas de pantalla reales de la ejecución

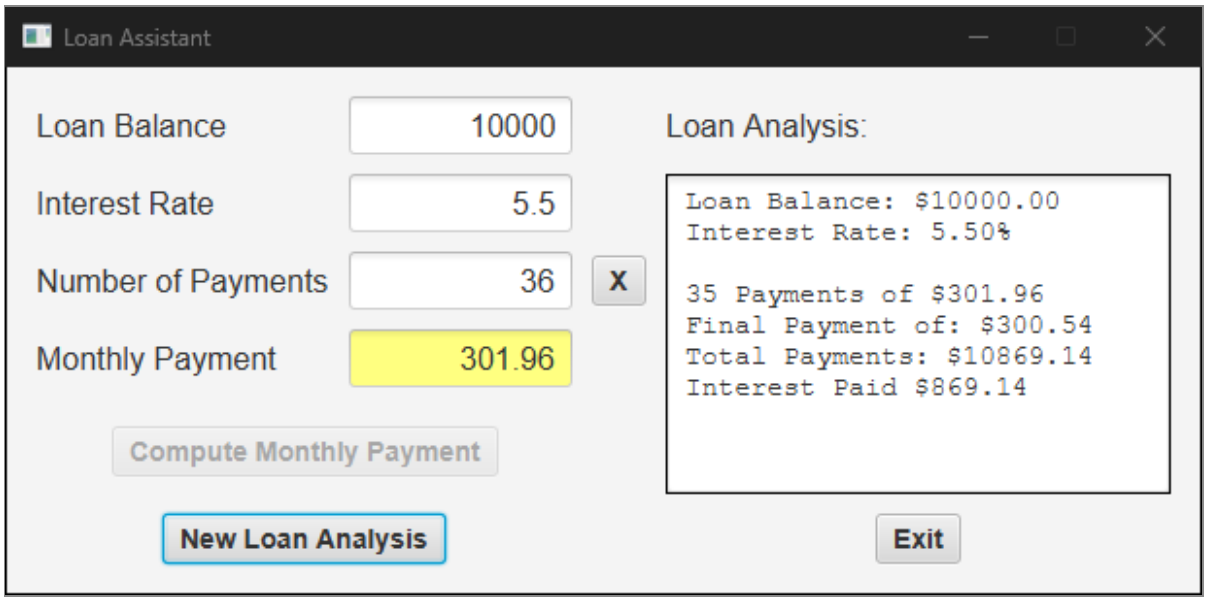


Figura 3.1. Modo "Compute Monthly Payment": préstamo de $10,000 a 5.5% durante 36 meses (pago mensual de $301.96 y pago final ajustado de $300.54).

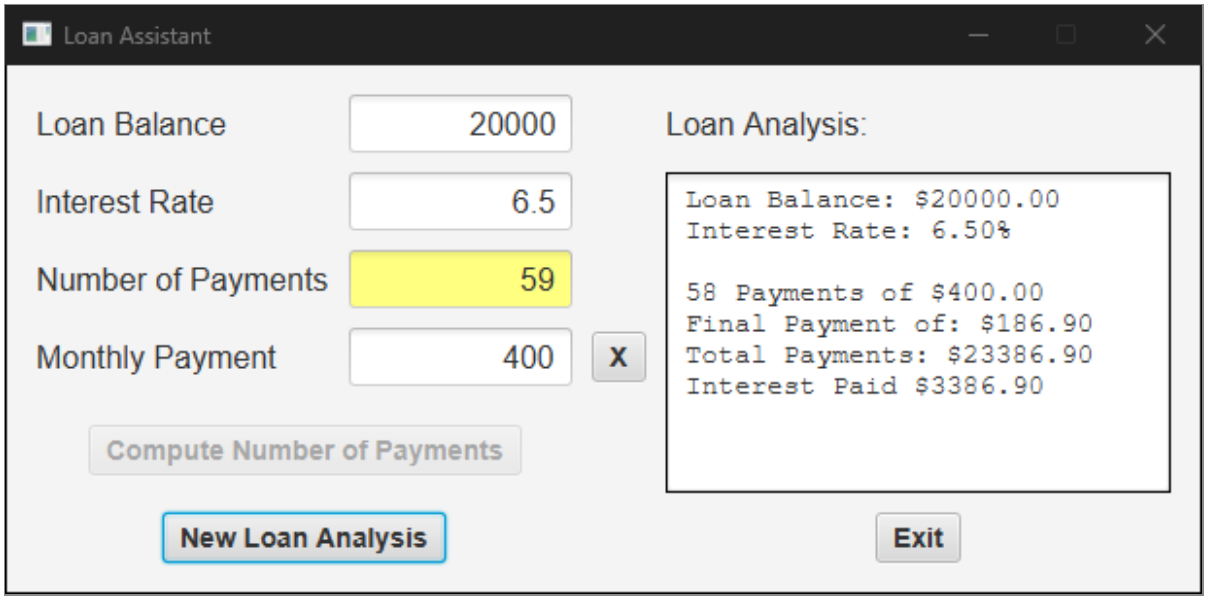


Figura 3.2. Modo "Compute Number of Payments": préstamo de $20,000 a 6.5% con pagos de $400.00 (58 pagos de $400.00 más un pago final de $186.90, dando 59 pagos en total).

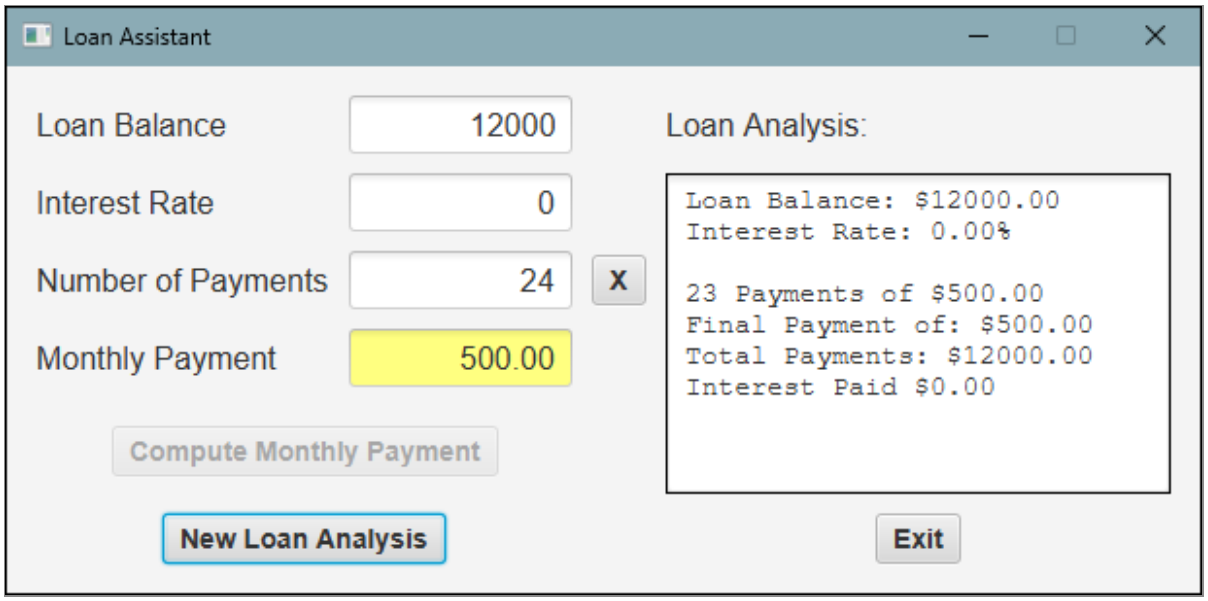


Figura 3.3. Prueba con tasa de interés del 0% ($12,000 a 24 meses = $500.00 mensuales sin error de división por cero).

4. Evidencia de funcionamiento

Se probaron los mismos casos de ejemplo que vienen en el manual del *Consumer Loan Assistant* para verificar que los resultados numéricos y las validaciones coincidan exactamente:

Caso de prueba	Datos de entrada	Resultado obtenido en la interfaz
1. Cálculo de mensualidad (Ejemplo del manual pág. 41)	Balance = `10000`, Interest = `5.5`, Months = `36`	Monthly Payment = `301.96`, 35 pagos de $301.96, pago final de $300.54, Total Payments = `$10869.14`, Interest Paid = `$869.14`.
2. Cálculo de número de pagos (Ejemplo del manual pág. 41)	Balance = `20000`, Interest = `6.5`, Payment = `400`	Number of Payments = `59` (58 pagos de $400.00 y pago final de $186.90), Total Payments = `$23386.90`, Interest Paid = `$3386.90`.
3. Tasa de interés 0% (Manual pág. 47)	Balance = `12000`, Interest = `0`, Months = `24`	Monthly Payment = `500.00`, Total Payments = `$12000.00`, Interest Paid = `$0.00`.
4. Validación de pago mínimo (Manual pág. 71-72)	Balance = `20000`, Interest = `5.5`, Payment = `10`	Muestra diálogo indicando que el pago mínimo debe ser `$92.00` y pregunta si se desea usar ese monto. Al elegir *Sí*, calcula `1230` pagos (1229 de $92.00 y final de $7.10).
5. Validación de entrada inválida o vacía	Dejar `Loan Balance` vacío o escribir letras	Muestra alerta *"Invalid or empty Loan Balance entry. Please correct."* y regresa el foco al campo con error.